Artificial Intelligence: Principles and Techniques

Mini Projects based on Stanford CS221 (Spring 2026) coursework

Theoretical Part

Written solutions to Assignments 1–7

Group 13

#	Full name	Student Number	Registration Number
1	MUJUNI INNOCENT	2400707155	24/U/07155/PS
2	**[TODO: name]**	**[TODO:]**	**[TODO:]**
3	**[TODO: name]**	**[TODO:]**	**[TODO:]**
4	**[TODO: name]**	**[TODO:]**	**[TODO:]**
5	**[TODO: name]**	**[TODO:]**	**[TODO:]**
6	**[TODO: name]**	**[TODO:]**	**[TODO:]**
7	**[TODO: name]**	**[TODO:]**	**[TODO:]**
8	**[TODO: name]**	**[TODO:]**	**[TODO:]**
9	**[TODO: name]**	**[TODO:]**	**[TODO:]**
10	**[TODO: name]**	**[TODO:]**	**[TODO:]**

The programming parts are in the companion notebook `CS221_Mini_Projects.ipynb`, which uses the same numbering.

November 2, 2026

Contents

Assignment 1: Foundations **2**
Problem 1: Optimization and probability . . . 3
Problem 2: Complexity . . . 5
Problem 3: Ethical issue spotting . . . 6
Problem 4: Programming . . . 7

Assignment 2: Sentiment Classification **7**
Problem 1: Building intuition . . . 8
Problem 2: Predicting movie ratings . . . 9
Problem 3: Sentiment classification . . . 10
Problem 4: Toxicity classification and maximum group loss . . . 11
Problem 5: K-means clustering . . . 13

Assignment 3: Route Planning **15**
Problem 1: Grid city . . . 16
Problem 2: Finding shortest paths . . . 17
Problem 3: Finding shortest paths with unordered waypoints . . . 19
Problem 4: Speeding up search with A* . . . 21

Assignment 4: Controlling MountainCar **21**
Problem 1: Value iteration . . . 22
Problem 2: Transforming MDPs . . . 22
Problem 3: Value iteration on Mountain Car . . . 23
Problem 4: Q-learning Mountain Car . . . 23
Problem 5: Safe exploration . . . 23

Assignment 5: Pac-Man **24**
Problem 1: Minimax . . . 25
Problem 2: Alpha-beta pruning . . . 25
Problem 3: Expectimax . . . 25
Problem 4: Evaluation function (extra credit) . . . 26
Problem 5: AI (mis)alignment and reward hacking . . . 26

Assignment 6: Bayesian Networks (Car Tracking) **27**
Problem 1: Emission probabilities . . . 28
Problem 2: Transition probabilities . . . 28
Problem 3: Which car is it? . . . 29
Problem 4: Ethics in advanced technologies . . . 30

Assignment 7: From Language to Logic **31**

Problem 1: Propositional logic. 32

Problem 2: First-order logic 32

Problem 3: Liar puzzle. 32

Problem 4: Odd and even integers 32

Problem 5: Semantic parsing (extra credit) 33

Problem 6: Explainability of logic-based systems 33

Problem 7: Applications of soundness and completeness 33

Assignment 1: Foundations

Problem 1: Optimization and probability

(1a) Weighted quadratic minimisation

Let $x_1, \dots, x_n \in \mathbb{R}$ and weights $w_1, \dots, w_n > 0$. For $f(\theta) = \sum_{i=1}^{n} w_i(\theta - x_i)^2$ (θ scalar), find the minimiser, show it is a minimum, and explain what goes wrong if some w_i are negative.

What we expect: Expression for the minimising θ and its derivation; a short argument that it is a minimum; 1–2 sentences on negative weights.

Solution. f is differentiable, so any minimiser is a critical point:

$$f'(\theta) = \sum_{i=1}^{n} 2w_i(\theta - x_i) = 2\theta \sum_i w_i - 2\sum_i w_i x_i = 0 \quad \Longrightarrow \quad \theta^\star = \frac{\sum_{i=1}^{n} w_i x_i}{\sum_{i=1}^{n} w_i},$$

the weighted average of the positions. It is a minimum because $f''(\theta) = 2\sum_i w_i > 0$ (all $w_i > 0$), so f is strictly convex and its unique critical point is the global minimum. (Division by $\sum_i w_i$ is safe because the sum is positive.)

If some w_i are negative, two things can break: $\sum_i w_i$ may be 0, so $\theta^\star$ is undefined, and if $\sum_i w_i < 0$ then $f'' < 0$, so the critical point is a *maximum* and $f(\theta) \to -\infty$ as $|\theta| \to \infty$, i.e. there is no minimum at all. Intuitively a negative weight "rewards" being far from x_i, so the problem is no longer about staying close to the points.

(1b) Swapping min and sum

Let $f(\mathbf{x}) = \min_{s \in [-1,1]} \sum_{i=1}^{d} s\, x_i$ and $g(\mathbf{x}) = \sum_{i=1}^{d} \min_{s_i \in [-1,1]} s_i x_i$ for $\mathbf{x} \in \mathbb{R}^d$. Which of $f \le g$, $f = g$, $f \ge g$ holds for all $\mathbf{x}$? Prove it.

What we expect: A short (3–5 line) proof using mathematical notation.

Solution. $f(\mathbf{x}) \ge g(\mathbf{x})$ for all $\mathbf{x}$. Write both as minimisations of the same quantity $\sum_i s_i x_i$:

$$f(\mathbf{x}) = \min_{\substack{s_1,\dots,s_d \in [-1,1] \\ s_1 = s_2 = \cdots = s_d}} \sum_{i=1}^{d} s_i x_i, \qquad g(\mathbf{x}) = \min_{s_1,\dots,s_d \in [-1,1]} \sum_{i=1}^{d} s_i x_i.$$

f minimises over the subset of $[-1,1]^d$ where all coefficients are equal, while g minimises the same objective over the whole cube. Minimising over a larger set can only give a smaller (or equal) value, so $g(\mathbf{x}) \le f(\mathbf{x})$. Concretely, $f(\mathbf{x}) = -|\sum_i x_i|$ (take $s = -\operatorname{sign} \sum_i x_i$) and $g(\mathbf{x}) = -\sum_i |x_i|$ (take $s_i = -\operatorname{sign} x_i$), and the triangle inequality $|\sum_i x_i| \le \sum_i |x_i|$ gives the same conclusion. The inequality is strict whenever the x_i do not all have the same sign, e.g. $\mathbf{x} = (1, -1)$: $f = 0 > g = -2$.

(1c) Expected points of a dice game

Roll a fair six-sided die until a 1 appears. Each 3 wins a points, each 6 loses b points, and $2, 4, 5$ change nothing. What is the expected number of points at the end, as a function of a, b?

What we expect: A recurrence for the value V, a short explanation of it, and the solved expression.

Solution. Let V be the expected number of points collected from now until the game stops. Each roll is independent, so after any roll that does not stop the game we are in exactly the same situation and expect V more points. Conditioning on the next roll:

$$V = \underbrace{\tfrac{1}{6}\cdot 0}_{\text{roll 1: stop}} + \underbrace{\tfrac{1}{6}(a+V)}_{\text{roll 3}} + \underbrace{\tfrac{1}{6}(-b+V)}_{\text{roll 6}} + \underbrace{\tfrac{3}{6}V}_{\text{roll 2,4,5}} = \frac{a-b}{6} + \frac{5}{6}V.$$

Solving, $\frac{1}{6}V = \frac{a-b}{6}$, hence $\boxed{V = a-b}$. (Sanity check: before the stopping 1 appears, each of the other five faces is equally likely, so on average there is exactly one 3 and one 6.)

(1d) Maximum likelihood for a coin

A coin with $P(\mathrm{H}) = p$, $0 < p < 1$, gives $\{\mathrm{T}, \mathrm{H}, \mathrm{H}, \mathrm{H}, \mathrm{H}\}$, so $L(p) = p^4(1-p)$. Which p maximises L? Prove it is a maximum and give an intuitive interpretation.

What we expect: The maximiser with its derivation (e.g. via $\log L$), proof that it is a maximum, and a 1-sentence interpretation.

Solution. log is strictly increasing, so L and $\ell(p) = \log L(p) = 4\log p + \log(1-p)$ have the same maximiser.

$$\ell'(p) = \frac{4}{p} - \frac{1}{1-p} = 0 \implies 4(1-p) = p \implies p^\star = \frac{4}{5}.$$

It is a maximum: $\ell''(p) = -\dfrac{4}{p^2} - \dfrac{1}{(1-p)^2} < 0$ for every $p \in (0,1)$, so ℓ is strictly concave and $p^\star$ is its unique global maximiser (equivalently, $L(p) > 0$ on $(0,1)$ while $L(p) \to 0$ at both ends, so the unique interior critical point must be the maximum).

Interpretation: $p^\star = 4/5$ is simply the empirical frequency of heads in the data (4 heads out of 5 flips); maximum likelihood says the most plausible coin is the one whose bias matches what we observed.

(1e) Conditional probability

Events A, B satisfy $P(A \mid B) = P(B \mid A)$, $P(A \cup B) = \frac{1}{3}$ and $P(A \cap B) > 0$. Prove that $P(A) > \frac{1}{6}$.

What we expect: A short (∼5 line) proof.

Solution. By the definition of conditional probability, $P(A \mid B) = \dfrac{P(A \cap B)}{P(B)}$ and $P(B \mid A) = \dfrac{P(A \cap B)}{P(A)}$. Setting them equal and using $P(A \cap B) > 0$ (so we may divide by it) gives $P(A) = P(B)$. By inclusion–exclusion,

$$\tfrac{1}{3} = P(A \cup B) = P(A) + P(B) - P(A \cap B) = 2P(A) - P(A \cap B),$$

so

$$P(A) = \frac{\frac{1}{3} + P(A \cap B)}{2} > \frac{\frac{1}{3} + 0}{2} = \frac{1}{6},$$

where the inequality is strict because $P(A \cap B) > 0$.

(1f) Gradient

For $\mathbf{w} \in \mathbb{R}^d$, constants $\mathbf{a}_i, \mathbf{b}_j \in \mathbb{R}^d$, $\lambda \in \mathbb{R}$ and $n \in \mathbb{N}$,

$$f(\mathbf{w}) = \left(\sum_{i=1}^{n} \sum_{j=1}^{n} (\mathbf{a}_i^\top \mathbf{w} - \mathbf{b}_j^\top \mathbf{w})^2 \right) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2.$$

Compute $\nabla f(\mathbf{w})$ (no summations over dimensions).

What we expect: A compact expression for the gradient and its derivation (∼5 lines).

Solution. Write each term as $(\mathbf{c}_{ij}^\top \mathbf{w})^2$ with $\mathbf{c}_{ij} = \mathbf{a}_i - \mathbf{b}_j$, since $\mathbf{a}_i^\top \mathbf{w} - \mathbf{b}_j^\top \mathbf{w} = (\mathbf{a}_i - \mathbf{b}_j)^\top \mathbf{w}$. Two standard gradients: $\nabla_{\mathbf{w}} (\mathbf{c}^\top \mathbf{w}) = \mathbf{c}$ and $\nabla_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|_2^2 = \mathbf{w}$. By the chain rule, $\nabla_{\mathbf{w}} (\mathbf{c}^\top \mathbf{w})^2 = 2(\mathbf{c}^\top \mathbf{w})\, \mathbf{c}$, so

$$\nabla f(\mathbf{w}) = 2 \sum_{i=1}^{n} \sum_{j=1}^{n} ((\mathbf{a}_i - \mathbf{b}_j)^\top \mathbf{w})(\mathbf{a}_i - \mathbf{b}_j) + \lambda \mathbf{w}.$$

Equivalently, with the $d \times d$ matrix $M = \sum_{i,j} (\mathbf{a}_i - \mathbf{b}_j)(\mathbf{a}_i - \mathbf{b}_j)^\top$, $f(\mathbf{w}) = \mathbf{w}^\top M \mathbf{w} + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$ and $\nabla f(\mathbf{w}) = (2M + \lambda I)\, \mathbf{w}$. For $d = 1$ this reduces to $f'(w) = 2 \sum_{i,j} (a_i - b_j)^2 w + \lambda w$, as expected.

Problem 2: Complexity

(2a) Placing three rectangles

On an $n \times n$ grid of points, how many ways are there to place 3 axis-aligned rectangles whose corners are grid points (degenerate and overlapping rectangles allowed)? Answer as $O(n^c)$ or $O(c^n)$.

What we expect: A big-O bound and ∼2 sentences of reasoning.

Solution. $O(n^{12})$. An axis-aligned rectangle is fully determined by the x-coordinates of its left and right sides and the y-coordinates of its bottom and top sides, each of which is one of n grid values (repeats allowed, giving degenerate rectangles), so there are $\Theta(n^2) \cdot \Theta(n^2) = \Theta(n^4)$ rectangles. The three rectangles are placed independently and may coincide, so the number of placements is $\Theta(n^4)^3 = \Theta(n^{12})$; whether the three are ordered or not only changes the constant (3!), not the exponent.

(2b) Minimum-cost monotone path

On an $n \times 2n$ grid, go from $(1, 1)$ to $(n, 2n)$ using only down/right steps. Each point has cost $c(i, j)$ (possibly negative, $O(1)$ to evaluate), and a path's cost is the sum over the points it visits. Give the most efficient algorithm for the minimum path cost and its runtime.

What we expect: Algorithm description (∼5 sentences), the big-O runtime, and why.

Solution. Use dynamic programming. Let $C(i, j)$ be the minimum cost of a path from $(1, 1)$ to (i, j). Because every step goes down or right, (i, j) can only be entered from $(i - 1, j)$ or $(i, j - 1)$, so

$$C(1, 1) = c(1, 1), \qquad C(i, j) = c(i, j) + \min\big(C(i - 1, j),\, C(i, j - 1)\big),$$

where a term with $i - 1 < 1$ or $j - 1 < 1$ is treated as $+\infty$ (so the first row and column just accumulate). Fill the table row by row (any order in which $(i - 1, j)$ and $(i, j - 1)$ are computed before (i, j) works), and return $C(n, 2n)$. Negative costs are not a problem: the move graph is acyclic, so no path can revisit a point and the recurrence remains valid.

Runtime $O(n^2)$: there are $n \cdot 2n = 2n^2$ cells and each costs $O(1)$ (one call to c and one min). This is optimal, since any correct algorithm must look at every $c(i, j)$ (each point lies on some path and a single very negative value could change the answer). Keeping only the previous row reduces memory to $O(n)$.

Problem 3: Ethical issue spotting

For each scenario, either identify one concern from the NeurIPS Ethics Guidelines (General Ethical Conduct / Potential Negative Social Impacts), copy it out and underline it, and justify it, or argue that there is no concern.

What we expect: A 2–5 sentence paragraph per scenario.

(3a) Loan-default prediction

A loan-default model relies mainly on location, accepts urban and rejects rural applicants, and has a significantly higher false-positive rate for Black and male applicants.

Solution. This model raises a *Discrimination / Bias and fairness* concern: it may encode, contain, or potentially exacerbate bias against people of a certain gender, race, sexuality, or who have other protected characteristics. A higher false-positive rate for Black and male applicants means that creditworthy people in those groups are denied loans more often than equally creditworthy people elsewhere, which is a direct, material harm tied to protected characteristics. Relying mainly on location makes this worse: residence is strongly correlated with race in many countries (historical "redlining"), so location acts as a proxy that lets the model discriminate even without a race feature, and the systematic rejection of rural applicants adds a geographic form of exclusion that can entrench existing inequality in access to credit.

(3b) Stylometry

Highly accurate ML stylometry predicts the authors of anonymous texts (including code) and could de-anonymise a wide range of writing.

Solution. The concern here is *Surveillance*: the technique could be used to facilitate surveillance, profiling, or the de-anonymization of individuals. Although the stated use is historical and literary scholarship, the same model applied to blog posts, forum comments or source code could unmask whistle-blowers, dissidents, journalists' sources or contributors to privacy-preserving software, and expose them to retaliation; anonymity is often the only protection such authors have. Because the capability is general and cheap to run at scale, the negative use is at least as likely as the benign one, so the researchers should discuss this dual use and consider mitigations (e.g. restricted release of models, research on authorship obfuscation as a countermeasure).

(3c) Celebrity face dataset

Millions of celebrity faces were scraped from Google Images, without permission and including copyrighted images, to build facial recognition; copyrighted images are distributed as URLs plus bounding boxes.

Solution. This violates the data-related rules of General Ethical Conduct, in particular *Consent*: datasets containing people should have been collected with the consent of the people depicted, and the *Copyright and Fair Use* requirement that researchers respect the copyright of the data. The subjects never agreed to have their faces used to train a biometric system, and being famous does not waive that interest. Distributing URLs plus bounding boxes instead of the images is a technicality that makes the copyrighted data just as usable while avoiding responsibility for it. There is also a downstream *Surveillance* concern, since facial recognition trained this way can be applied to anyone, not only to celebrities.

(3d) Plant-species identification

A model identifies plant species from a photo. It was trained on iNaturalist photos whose users consented to research use, and it is only used inside the app.

Solution. We do not think this scenario raises an ethical concern under the guidelines. The data were contributed by users who explicitly consented to the use of their photos for research, so the consent and privacy requirements are satisfied; the subjects are plants rather than people, so there is no protected-group bias to encode; and the deployment is narrow (identification inside the same app, at the user's request), which limits the risk of misuse or negative side effects. The only residual risk we see is indirect: precise photo locations of rare or protected species could help poachers, but that is a property of the public observation data and the app's location policy, not of the identification model itself.

Problem 4: Programming

*Programming part. The implementation, documentation and test output are in the notebook, section **1.4a–1.4f**.*

Functions: `find_alphabetically_first_word`, `euclidean_distance`, `mutate_sentences`, `sparse_vector_dot_product`, `increment_sparse_vector`, `find_nonsingleton_words`. All six pass the local grader (8/8 points); the hidden tests run without errors or timeouts.

Assignment 2: Sentiment Classification

Problem 1: Building intuition

Mini-reviews: (-1) *not good*; (-1) *pretty bad*; $(+1)$ *good plot*; $(+1)$ *pretty scenery.* $\phi(x)$ = word counts and $\text{Loss}_{\text{hinge}}(x, y, \mathbf{w}) = \max\{0,\, 1 - \mathbf{w} \cdot \phi(x)\, y\}$.

(1a) Four SGD steps

Run one SGD update per review, in the order given, with $\eta = 0.1$ from $\mathbf{w} = \mathbf{0}$ (gradient $= 0$ when the margin is exactly 1). Give the final weights of (*pretty, good, bad, plot, not, scenery*).

What we expect: The weight vector in that order, e.g. $[0.1, 0.2, 0.3, 0.4, 0.5, 0.6]$.

Solution. The hinge loss has gradient $\nabla_{\mathbf{w}}\text{Loss} = -y\,\phi(x)$ when the margin $\mathbf{w} \cdot \phi(x)\, y < 1$ and 0 otherwise, so each SGD step is $\mathbf{w} \leftarrow \mathbf{w} + \eta\, y\, \phi(x)$ *if* the margin is below 1, and no change otherwise. Weights are listed in the order (pretty, good, bad, plot, not, scenery).

step	review (y)	margin	update	weights after the step
0	—	—	—	$[0,\ 0,\ 0,\ 0,\ 0,\ 0]$
1	not good (-1)	$0 < 1$	$\mathbf{w} \mathrel{-}= 0.1\,\phi$	$[0,\ -0.1,\ 0,\ 0,\ -0.1,\ 0]$
2	pretty bad (-1)	$0 < 1$	$\mathbf{w} \mathrel{-}= 0.1\,\phi$	$[-0.1,\ -0.1,\ -0.1,\ 0,\ -0.1,\ 0]$
3	good plot $(+1)$	$-0.1 < 1$	$\mathbf{w} \mathrel{+}= 0.1\,\phi$	$[-0.1,\ 0,\ -0.1,\ 0.1,\ -0.1,\ 0]$
4	pretty scenery $(+1)$	$-0.1 < 1$	$\mathbf{w} \mathrel{+}= 0.1\,\phi$	$[0,\ 0,\ -0.1,\ 0.1,\ -0.1,\ 0.1]$

(Margins: steps 1–2 have $\mathbf{w} \cdot \phi = 0$ because no word has been seen yet; in steps 3–4 $\mathbf{w} \cdot \phi = -0.1 + 0$ since *good* and *pretty* carry -0.1 and the other word is new.) Final weights: $\boxed{[\,0,\ 0,\ -0.1,\ 0.1,\ -0.1,\ 0.1\,]}$ for (pretty, good, bad, plot, not, scenery).

(1b) A non-separable dataset

Dataset: (-1) *bad*; $(+1)$ *good*; $(+1)$ *not bad*; (-1) *not good.* Prove that no linear classifier $f_{\mathbf{w}}(x) = \text{sign}(\mathbf{w} \cdot \phi(x))$ on word-count features achieves zero error, then propose one extra feature that fixes this.

What we expect: A short proof (∼3–5 sentences) and a viable additional feature.

Solution. **Proof.** With word-count features the four scores are $s_1 = w_{\text{bad}}$, $s_2 = w_{\text{good}}$, $s_3 = w_{\text{not}} + w_{\text{bad}}$ and $s_4 = w_{\text{not}} + w_{\text{good}}$. Zero error needs s_2, s_3 to have the sign of $+1$ and s_1, s_4 the sign of -1, so in particular $s_2 + s_3 > s_1 + s_4$ (the left side is the sum of two scores classified positive, the right side of two scores classified negative, and with any convention for sign(0) the two sides cannot be equal while being classified differently). But $s_2 + s_3 = w_{\text{good}} + w_{\text{not}} + w_{\text{bad}} = s_1 + s_4$, a contradiction. Hence no weight vector separates the data: the effect of "not" is additive in a linear model, so it must push both reviews in the same direction, whereas the data require it to flip the sentiment.

Fix. Add one feature that lets the model see the interaction, e.g. the bigram indicator $\phi_{\text{not bad}}(x) = \mathbf{1}[\text{"not bad" occurs in } x]$. Then, for instance, $w_{\text{bad}} = -1$, $w_{\text{good}} = 1$, $w_{\text{not}} = -2$,

$w_{\text{not bad}} = 4$ gives scores $-1,\ 1,\ -1-2+4=1,\ 1-2=-1$, which classifies all four reviews correctly. (Any "not X" bigram feature, or a feature for a negation word immediately preceding a sentiment word, works the same way.)

Problem 2: Predicting movie ratings

Predictor $\sigma(\mathbf{w}\cdot\phi(x))$ with $\sigma(z)=(1+e^{-z})^{-1}$, rating $y\in[0,1]$. (No Wolfram Alpha or similar.)

(2a) Squared loss

Write $\text{Loss}(x,y,\mathbf{w})$ for squared loss on a single data point.

What we expect: A mathematical expression (may use σ).

Solution.

$$\text{Loss}(x,y,\mathbf{w}) = \big(\sigma(\mathbf{w}\cdot\phi(x)) - y\big)^2.$$

(2b) Its gradient

Compute $\nabla_{\mathbf{w}}\text{Loss}(x,y,\mathbf{w})$ in terms of $p=\sigma(\mathbf{w}\cdot\phi(x))$.

What we expect: A mathematical expression for the gradient.

Solution. The logistic function satisfies $\sigma'(z)=\sigma(z)(1-\sigma(z))$. With $p=\sigma(\mathbf{w}\cdot\phi(x))$ and the chain rule ($\nabla_{\mathbf{w}}(\mathbf{w}\cdot\phi(x))=\phi(x)$):

$$\nabla_{\mathbf{w}}\text{Loss}(x,y,\mathbf{w}) = 2\,(p-y)\cdot\sigma'(\mathbf{w}\cdot\phi(x))\cdot\phi(x) = 2\,(p-y)\,p\,(1-p)\,\phi(x).$$

(2c) Vanishing gradients

For one data point with $y=1$, under what conditions on $\mathbf{w}$ does $\|\nabla_{\mathbf{w}}\text{Loss}\|_2$ become arbitrarily small? Can it be exactly zero ($\|\mathbf{w}\|_2$ large but finite)?

What we expect: 1–2 sentences on the conditions; 1–2 sentences on whether it can be exactly zero.

Solution. With $y=1$ the gradient is $-2\,(1-p)^2\,p\,\phi(x)$, whose magnitude $2(1-p)^2p\,\|\phi(x)\|_2$ goes to 0 when $p\to 1$, i.e. when $\mathbf{w}\cdot\phi(x)\to+\infty$ (take $\mathbf{w}=c\,\phi(x)$ with $c\to\infty$: the prediction is correct and very confident), *and also* when $p\to 0$, i.e. $\mathbf{w}\cdot\phi(x)\to-\infty$ (the prediction is confidently *wrong*; this is the vanishing-gradient trap, because the loss is near its maximum 1 yet gradient descent barely moves).

It can never be exactly zero for finite $\mathbf{w}$ (assuming $\phi(x)\neq\mathbf{0}$): σ takes values strictly inside $(0,1)$, so $p\in(0,1)$ and $(1-p)^2p>0$. The gradient is only zero in the limit $\|\mathbf{w}\|_2\to\infty$ (or trivially if $\phi(x)=\mathbf{0}$).

Problem 3: Sentiment classification

*Programming part. The implementation, documentation and test output are in the notebook, section **2.3a–2.3d**.*

`extract_word_features`, `learn_predictor` (SGD on hinge loss), `generate_dataset`, `extract_character_features`. With word features, 20 epochs and $\eta = 0.01$ the learned predictor reaches a train error of 2.76% and a validation error of 27.35% (targets: $< 4\%$ and $< 30\%$). All local tests pass (9/9 points). (All numbers in this section were produced with Python 3.12, the graders' version; Python 3.11 gives slightly different values because 3.12 changed how `sum()` rounds floats.)

(3e) Character n-grams

Run the linear predictor with `extract_character_features` for several n. Which n gives the smallest validation error, and why is it nearly as good as word features? Construct a one-sentence review for which character n-grams probably beat word features, and explain why.

What we expect: A paragraph (∼4–6 sentences) with the best n and the reason; a one-sentence review with an explanation.

Solution. We trained with the same settings as 3b (20 epochs, $\eta = 0.01$) for $n = 1, \ldots, 10$ (notebook §2.3e):

n	train error	validation error	# features
1	0.4572	0.4789	76
2	0.2839	0.3953	1 465
3	0.0025	0.3210	10 814
4	0.0000	0.2839	46 541
5	0.0000	0.2724	107 308
6	0.0000	0.2735	166 482
7	0.0003	**0.2712**	212 442
8	0.0006	0.2932	243 425
9	0.0008	0.3092	261 895
10	0.0008	0.3376	273 322
words	0.0276	0.2735	—

The smallest validation error, 27.12%, is obtained with $n = 7$, marginally better than word features (27.35%); $n = 5$–7 are all within a quarter of a percentage point of each other. This is the regime in which an n-gram is about the length of an English word, so most n-grams are essentially a word stem together with a piece of its neighbours: the features carry the same sentiment information as words ("awful", "superb"), while also sharing evidence between inflections and derivations (*bore*/*boring*/*bored*) and capturing short word-boundary contexts such as *notgoo*. Very small n performs poorly because 1- and 2-grams are shared by almost every word and carry little meaning; very large n overfits, because each feature is nearly a unique word pair that rarely reappears in the validation set (train error is ≈ 0 from $n = 4$ on while validation error grows for $n \geq 8$).

Example review: "*Dissapointing, unwatchable and laughably overacted.*" Character n-grams probably beat word features here: the misspelt *dissapointing* and the rare derived words *unwatchable*, *laughably*, *overacted* are unlikely to appear verbatim in the training vocabulary,

so a word model has weight 0 for them, whereas their n-grams (*appoint*, *watch*, *laugh*, *overact*) overlap with n-grams of common negative words seen in training.

Problem 4: Toxicity classification and maximum group loss

$\phi(x) = [1, d, t]$: bias, demographic mention $d \in \{0, 1\}$, toxic words $t \in \{0, 1\}$; label $y \in \{-1, 1\}$ (1 = toxic). Groups are defined by (y, d). Zero-one loss; $\text{TrainLoss}_{\max}(\mathbf{w}) = \max_g \text{TrainLoss}_g(\mathbf{w})$.

y	d	t	# points
-1	0	0	63
-1	0	1	27
-1	1	0	7
-1	1	1	3
1	0	0	3
1	0	1	7
1	1	0	27
1	1	1	63
	Total		200

Classifier D: $\mathbf{w} = [-0.1, 1, 0]$ Classifier T: $\mathbf{w} = [-0.1, 0, 1]$

(4a) Behaviour of D and T

Describe in words how each classifier behaves.

What we expect: For each classifier, an "if and only if" statement for when $f_{\mathbf{w}}(x) = 1$.

Solution. **Classifier D:** $\mathbf{w} \cdot \phi(x) = -0.1 + d$, which is positive exactly when $d = 1$. So $f_D(x) = 1$ *if and only if* the comment mentions a demographic identity ($d = 1$): D flags every comment that mentions an identity as toxic and nothing else, ignoring the actual words used.

Classifier T: $\mathbf{w} \cdot \phi(x) = -0.1 + t$, so $f_T(x) = 1$ *if and only if* the comment contains a toxic word ($t = 1$): T flags exactly the comments with toxic vocabulary, ignoring identity mentions. (The bias -0.1 only ensures the score is never exactly 0.)

(4b) Losses of classifier D

Compute D's average loss, its per-group losses, and its maximum group loss.

What we expect: The average loss, the completed table below, and the maximum group loss.

Solution. D predicts $y = 1$ iff $d = 1$, so it is wrong on every example with $y \neq (2d - 1)$: the non-toxic comments with identity mentions ($y = -1, d = 1$: $7 + 3 = 10$) and the toxic comments without them ($y = 1, d = 0$: $3 + 7 = 10$).

	$y = 1$	$y = -1$
$d = 1$	$0/90 = 0$	$10/10 = 1$
$d = 0$	$10/10 = 1$	$0/90 = 0$

Average loss: $\text{TrainLoss}_{\text{avg}}(D) = \dfrac{10+10}{200} = 0.1$. Maximum group loss: $\text{TrainLoss}_{\max}(D) = 1.0$ (two whole groups, the minority groups, are misclassified completely).

(4c) Losses of classifier T; comparison

Compute the same three quantities for T. Which classifier has the lower average loss, and which has the lower maximum group loss?

What we expect: The average loss, completed table, and maximum group loss, plus both comparisons.

Solution. T predicts $y = 1$ iff $t = 1$; within every (y, d) group exactly 30% of the points have the "wrong" t.

	$y = 1$	$y = -1$
$d = 1$	$27/90 = 0.3$	$3/10 = 0.3$
$d = 0$	$3/10 = 0.3$	$27/90 = 0.3$

Average loss: $\text{TrainLoss}_{\text{avg}}(T) = \dfrac{27+3+3+27}{200} = 0.3$. Maximum group loss: $\text{TrainLoss}_{\max}(T) = 0.3$.

Comparison: **D has the lower average loss** ($0.1 < 0.3$) because it exploits the spurious identity–toxicity correlation that holds for 90% of the data, but **T has the lower maximum group loss** ($0.3 < 1.0$) because its errors are spread evenly across groups instead of falling entirely on the two minority groups.

(4d) Fairness and the mixed objective

$\text{TrainLoss}_{\text{new}}(\mathbf{w}) = \lambda\,\text{TrainLoss}_{\text{avg}}(\mathbf{w}) + (1-\lambda)\,\text{TrainLoss}_{\max}(\mathbf{w})$, with $\lambda \in [0, 1]$. Use the ethical frameworks from the *Algorithms and Distribution* module.

(i) $\lambda = 1$: describe the optimal classifier and the framework it aligns with.

What we expect: 1–2 sentences, justified by the framework.

Solution. With $\lambda = 1$ the objective is the plain average loss, so the optimal classifier is the one that makes the fewest mistakes overall regardless of who bears them, here D-like behaviour (loss 0.1). This is the *utilitarian* principle: maximise aggregate welfare (total number of correctly treated commenters), even though it leaves the minority groups ($y = -1, d = 1$ and $y = 1, d = 0$) completely misclassified.

(ii) $\lambda = 0$: same question.

What we expect: 1–2 sentences, justified by the framework.

Solution. With $\lambda = 0$ only the worst-off group matters, so the optimal classifier equalises and minimises the largest group loss, here T (every group at 0.3). This is the *Rawlsian maximin* (difference) principle: judge a rule by how it treats the least-advantaged group and accept a lower average to improve their outcome.

(iii) $\lambda = 0$ with a new training set in which group $(y = 1, d = 1)$ has size 1 (the other groups have 65, 65, 69). How would you factor group size into $\text{TrainLoss}_{\max}$?

What we expect: The adjustment, justified in 1–2 sentences.

Solution. Weight each group's loss by how reliably it can be estimated, e.g. discount groups below a minimum size m: $\text{TrainLoss}'_{\max}(\mathbf{w}) = \max_g \min(1, \frac{|D_{\text{train}}(g)|}{m})\, \text{TrainLoss}_g(\mathbf{w})$ (or equivalently add pseudo-counts to each group's error rate). A single example gives a loss of either 0 or 1 and says almost nothing about the group, so without such a correction one noisy point would dictate the whole objective and let an unrepresentative group override the three groups with 65–69 members.

(iv) What λ would you deploy on a real platform that cares equally about average and maximum group loss? Discuss the trade-offs with reference to the frameworks.

What we expect: 3–5 sentences.

Solution. Caring equally about the two terms means $\lambda = 0.5$. On our data this gives $\text{TrainLoss}_{\text{new}}(D) = 0.5{\cdot}0.1 + 0.5{\cdot}1.0 = 0.55$ and $\text{TrainLoss}_{\text{new}}(T) = 0.5{\cdot}0.3 + 0.5{\cdot}0.3 = 0.30$, so T would be deployed. The trade-off is utilitarian efficiency against Rawlsian protection of the worst-off: T triples the average error (more toxic comments slip through and more ordinary commenters are flagged), but D would flag *every* comment in which someone states their identity ("I'm a woman in computer science"), silencing exactly the groups that moderation is meant to protect and encoding the spurious correlation as policy. A mixed λ therefore accepts some loss of aggregate accuracy in exchange for no group being systematically harmed, which is also the more defensible position for a platform that must justify its decisions to all its users.

Problem 5: K-means clustering

$\phi(x_1) = [0, 0]$, $\phi(x_2) = [4, 0]$, $\phi(x_3) = [6, 0]$, $\phi(x_4) = [11, 0]$.

(5a) 2-means by hand

Run 2-means to convergence from (i) $\mu_1 = [0, 0],\ \mu_2 = [11, 0]$ and (ii) $\mu_1 = [0, 0],\ \mu_2 = [4, 0]$. Give the assignments z, centres μ and final total loss.

What we expect: For each initialisation: centres and assignments at every step, and the final loss.

Solution. All points lie on the x-axis, so we write only x-coordinates. Each step assigns every point to the nearest centre, then moves each centre to the mean of its points; the loss is $\sum_i \|\phi(x_i) - \mu_{z_i}\|_2^2$.

(i) $\mu_1 = 0,\ \mu_2 = 11$**.**

- Step 1, assign: x_1=0 $\to \mu_1$ ($0 < 11$), x_2=4 $\to \mu_1$ ($4 < 7$), x_3=6 $\to \mu_2$ ($6 > 5$), x_4=11 $\to \mu_2$; $z = (1, 1, 2, 2)$. Update: $\mu_1 = \frac{0+4}{2} = 2$, $\mu_2 = \frac{6+11}{2} = 8.5$.
- Step 2, assign: $0 \to \mu_1$ ($2 < 6.5$), $4 \to \mu_1$ ($2 < 4.5$), $6 \to \mu_2$ ($4 > 2.5$), $11 \to \mu_2$; $z = (1, 1, 2, 2)$ unchanged $\Rightarrow$ converged.

Final: $z = (1, 1, 2, 2)$, $\mu_1 = [2, 0]$, $\mu_2 = [8.5, 0]$, loss $= 2^2 + 2^2 + 2.5^2 + 2.5^2 = 4 + 4 + 6.25 + 6.25 =$ **20.5**.

(ii) $\mu_1 = 0,\ \mu_2 = 4$**.**

- Step 1, assign: $0 \to \mu_1$, $4 \to \mu_2$ ($0 < 4$), $6 \to \mu_2$ ($2 < 6$), $11 \to \mu_2$; $z = (1, 2, 2, 2)$. Update: $\mu_1 = 0$, $\mu_2 = \frac{4+6+11}{3} = 7$.
- Step 2, assign: $0 \to \mu_1$, $4 \to \mu_2$ ($3 < 4$), $6 \to \mu_2$ ($1 < 6$), $11 \to \mu_2$ ($4 < 11$); $z = (1, 2, 2, 2)$ unchanged $\Rightarrow$ converged.

Final: $z = (1, 2, 2, 2)$, $\mu_1 = [0, 0]$, $\mu_2 = [7, 0]$, loss $= 0 + 3^2 + 1^2 + 4^2 =$ **26**.

The two initialisations converge to different local optima; (i) is the better one, which illustrates why k-means is usually restarted from several random initialisations.

(5b) Implementing k-means

*Programming part. The implementation, documentation and test output are in the notebook, section **2.5b**.*

The implementation caches $\|\phi(x_i)\|_2^2$ for every example and $\|\mu_j\|_2^2$ for every centre once per epoch, so each distance $\|\phi(x_i) - \mu_j\|_2^2 = \|\phi(x_i)\|_2^2 - 2\,\phi(x_i) \cdot \mu_j + \|\mu_j\|_2^2$ costs one sparse dot product; it stops as soon as the assignments stop changing.

(5c) Scaling invariance

If every dimension of the data and the initial centroids is scaled by the same non-zero factor, are the same clusters guaranteed? What if only some dimensions are scaled?

What we expect: For each sub-question: a short explanation if the answer is yes, or a concrete numerical counterexample if no.

Solution. All dimensions scaled by the same $c \neq 0$: yes. Every squared distance is multiplied by the same constant c^2, $\|c\mathbf{x} - c\boldsymbol{\mu}\|_2^2 = c^2\|\mathbf{x} - \boldsymbol{\mu}\|_2^2$, so the nearest centre of every point (the $\arg\min$) is unchanged, and the new centres are the means of the same sets of points, hence exactly c times the old centres. By induction every iteration produces the same assignments as in the unscaled run, so the final clusters are identical (and the loss is simply c^2 times the original).

Only some dimensions scaled: no. Take $K = 2$ and the four points $A = (0, 0)$, $B = (0, 2)$, $C = (3, 0)$, $D = (3, 2)$ with initial centres $\mu_1 = A$, $\mu_2 = D$. Unscaled: B is at distance 2 from A and 3 from D, C is at distance 3 from A and 2 from D, so the clusters are $\{A, B\}$ and $\{C, D\}$ (centres $(0, 1)$ and $(3, 1)$, stable on the next pass). Now scale only the first coordinate by $\frac{1}{3}$: $A' = (0, 0)$, $B' = (0, 2)$, $C' = (1, 0)$, $D' = (1, 2)$, with the scaled initial centres $\mu_1 = A'$, $\mu_2 = D'$.

Now B' is at distance 2 from A' but 1 from D', and C' is at distance 1 from A' but 2 from D', so the clusters become $\{A', C'\}$ and $\{B', D'\}$ (centres $(0.5, 0)$ and $(0.5, 2)$, stable). The same points end up in different clusters, so non-uniform scaling changes the result; this is why features are normalised before clustering.

Assignment 3: Route Planning

Problem 1: Grid city

Infinite grid of integer locations (x, y), with $m, n \geq 0$:

- $s_{\text{start}} = (0, 0)$, $\quad \text{IsEnd}(s) = \mathbf{1}[s = (m, n)]$
- $\text{Actions}(s) = \{(+1, 0), (-1, 0), (0, +1), (0, -1)\}$, $\quad \text{Succ}(s, a) = s + a$
- $\text{Cost}((x, y), a) = 1 + \max(x, 0)$

(1a) Minimum cost

What is the minimum cost of reaching (m, n) from $(0, 0)$? Describe a path achieving it. Is it unique?

What we expect: An expression for the minimum cost; 1–2 sentences describing one optimal path and whether it is unique.

Solution. Minimum cost

$$\boxed{n + \sum_{x=0}^{m-1} (1 + x) = n + m + \frac{m(m-1)}{2} = n + \frac{m(m+1)}{2}}.$$

Every path must contain at least m east moves and n north moves. A north (or any) move costs 1 when taken at $x \leq 0$ and $1 + x \geq 2$ when taken at $x \geq 1$, so all north moves should be made before leaving $x = 0$; the k-th east move necessarily starts at $x = k - 1$ and costs $1 + (k - 1)$. Detours (west, south, or extra east/west pairs) only add moves of cost ≥ 1, so they never help. One optimal path is therefore: n steps north from $(0, 0)$ to $(0, n)$, then m steps east to (m, n). It is unique whenever $m \geq 1$ and $n \geq 1$: any north move taken at $x \geq 1$ costs at least 2 instead of 1, and moving to $x < 0$ first costs two extra unit moves, so no other path attains the minimum. (If $m = 0$ or $n = 0$ the straight line is trivially the only optimum.)

(1b) UCS on this problem

True or false: (i) UCS never terminates because there are infinitely many states. (ii) UCS returns the minimum-cost path and explores only (x, y) with $0 \leq x \leq m$, $0 \leq y \leq n$. (iii) UCS returns the minimum-cost path and explores only locations whose past cost is $\leq$ the minimum cost to (m, n).

What we expect: T/F for each sub-part.

Solution.

(i) False. All costs are ≥ 1, so only finitely many locations have past cost at most the (finite) optimal cost $C^\star$. UCS pops states in increasing order of past cost, so it reaches the goal and stops after exploring finitely many states; an infinite state space is not a problem as long as positive costs bound the number of states cheaper than the goal.

(ii) False. UCS explores every state whose past cost is below $C^\star$, and many of them lie outside the rectangle: $(-1, 0)$ has past cost 1, $(0, n + 1)$ has past cost $n + 1 \leq C^\star$ (for $m \geq 1$), and

all of $(-k, y)$ with $k + y < C^\star$ are explored too. UCS does return the minimum-cost path, but the second half of the statement is wrong.

(iii) True. UCS pops states from the frontier in non-decreasing order of past cost and terminates when it pops the goal (at past cost $C^\star$); hence every explored state has past cost $\leq C^\star$, and the returned path is optimal because all costs are non-negative.

(1c) UCS on an arbitrary graph

True or false: (i) Adding a connection between two nodes cannot increase the minimum cost between any start and end node. (ii) Making an action's cost small enough (possibly negative) guarantees that it appears in the path UCS returns, if a path exists. (iii) Increasing every action's cost by 1 does not change the minimum-cost path, only its cost.

What we expect: T/F for each sub-part.

Solution.

(i) True. Adding a connection only adds new paths; every path that existed before still exists with the same cost, so the minimum over the (larger) set of paths cannot increase.

(ii) False. Two reasons. The cheap action may not lie on *any* start–end path (e.g. an edge into a dead end, or one pointing away from the goal that can only be followed by expensive moves back), so no path containing it is ever returned. And if the cost is made negative, UCS is no longer correct at all: it assumes non-negative costs, so it may return a path without the cheap edge even when a cheaper path using it exists (or, with a negative cycle, there is no finite minimum).

(iii) False. Adding 1 to every action penalises paths with more actions. Example: path P_1 has three edges of cost 1 (total 3) and path P_2 one edge of cost 4. Originally P_1 is optimal ($3 < 4$); after the change P_1 costs 6 and P_2 costs 5, so the minimum-cost path switches to P_2.

Problem 2: Finding shortest paths

(2a) `ShortestPathProblem`

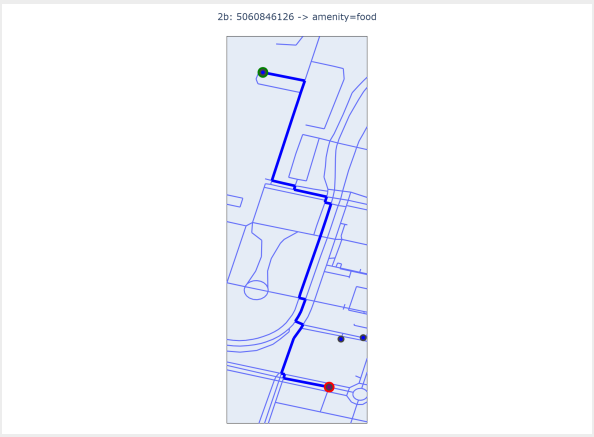
*Programming part. The implementation, documentation and test output are in the notebook, section **3.2a**.*

(2b) Stanford shortest-path experiments

Choose start locations and end tags with `get_stanford_shortest_path_problem`, and visualise them. Give (i) a setting that gave new insight into travelling around campus (was the system useful?) and (ii) a setting where the minimum-cost path is *not* desirable. Is that caused by modelling assumptions (missing tags or paths, inaccurate locations or distances)?

What we expect: Screenshots of both visualisations; one or more sentences on something learned about travelling, and something incorrect about the map or the modelling assumptions.

Solution. Both routes were produced by `ShortestPathProblem` + UCS on the Stanford map and drawn with the provided `visualization.py` (exported to PNG by `scripts/route_figures.py`). Red marker = start, green marker = end; blue dots are `amenity` locations and purple dots landmarks.



(a) `landmark=gates` → `amenity=food`: 659 m, 38 locations; ends at the Cantor Arts Center café.



(b) `landmark=hoover_tower` → `amenity=food`: 447 m walked to a café that is only 137 m away in a straight line.

Figure 1: Problem 2b: shortest paths on the Stanford map.

(i) New insight. Our custom problem (`get_stanford_shortest_path_problem`) asks for the nearest place with food when leaving the Gates building. We expected Coupa Café at Green Library or Tresidder, but the search returns the café at the Cantor Arts Center, 659 m away, which is closer than either (Tresidder is >700 m). The system is useful precisely because the end is given as a *tag* rather than a fixed destination: it searched all four food locations at once and found an option we would not have thought of.

(ii) Undesirable path. From Hoover Tower the nearest food is Coupa Café at Green Library, 137 m away as the crow flies, yet the minimum-cost path is 447 m long (3.3× the straight-line distance) and loops around the surrounding buildings. This is a modelling problem, not a search problem: the map only contains the footpaths present in the OpenStreetMap extract (and filters out ways tagged as not walkable), so the direct paths across the Hoover courtyard and through the library arcade are simply missing, and every landmark is snapped to the nearest OSM node within 250 m, which can put it on the "wrong" side of a building. A second, smaller issue is the tags themselves: Stanford Stadium is tagged `amenity=food` although its concessions only operate on game days, so a hungry person at the stadium would be told they have already arrived (a 0 m path), and no tag carries opening hours or whether a place is indoors.

(2c) Negative externalities

What negative externalities could follow from wide deployment of this shortest-path system? Discuss the effects on (1) users and (2) non-users, and ways to reduce the mismatch between the model and the real world (see "Why Traffic Apps Make Congestion Worse", "The Perfect Selfishness of Mapping Apps", and the Externalities module).

What we expect: 3–5 sentences: two externalities (one for users, one for non-users) and at least one mitigation.

Solution. Our model treats each traveller in isolation on a static map, but in reality every routed person changes the conditions for everyone else. *For users:* if a large fraction of people receive the same "shortest" route, that route becomes congested, so the predicted travel time is wrong and users end up slower than the system promised (the "selfish routing" effect described in the articles, where individually optimal choices produce a collectively worse equilibrium). *For non-users:* the shortest routes often cut through quiet residential streets, narrow campus paths or bike lanes, so residents, pedestrians and cyclists who never asked for the system absorb more traffic, noise, pollution and accident risk, while the costs of their roads being used are not counted anywhere in our objective. The root cause is the mismatch between the model (fixed edge costs, independent travellers) and the world (costs that depend on load and on externalities). Ways to reduce it: make edge costs depend on predicted load and update them from live data; add the social cost of a path (residential streets, school zones, emissions) to the cost function instead of only distance; and deliberately spread users over several near-optimal routes rather than sending everyone down the same one.

Problem 3: Finding shortest paths with unordered waypoints

(3a) `WaypointsShortestPathProblem`

*Programming part. The implementation, documentation and test output are in the notebook, section **3.3a**.*

(3b) Size of the state space

With n locations and k waypoint tags, what is the maximum number of states UCS could visit under a suitable state definition for 3a?

What we expect: An expression in n and k with a brief justification.

Solution. At most $\boxed{n \cdot 2^k}$ states. Our state is (current location, set of waypoint tags covered so far). The location takes one of n values and the covered set is one of the 2^k subsets of the k waypoint tags, so there are $n \cdot 2^k$ distinct states and UCS can visit each at most once. Nothing else about the past (the exact locations visited, their order, the distance travelled) affects which future paths are feasible or their costs, so this is the smallest sufficient memory; in practice far fewer states are reached, because tags are only picked up at the few locations that carry them.

(3c) Stanford waypoints experiment

Choose a start location, waypoint tags (at least two) and an end tag with `get_stanford_waypoints_shortest_path_problem`, and visualise the route. What is unexpected or interesting about it? Does it match your expectations, and what important factors does it fail to capture?

What we expect: A screenshot, the list of waypoint tags, and one or more sentences of discussion.

Solution. Our custom problem (`get_stanford_waypoints_shortest_path_problem`) is an end-of-day errand run: start at the Rains graduate housing, cover the waypoint tags `amenity=food`, `landmark=bookstore` and `landmark=green_library` in any order, and end at `landmark=gates`.

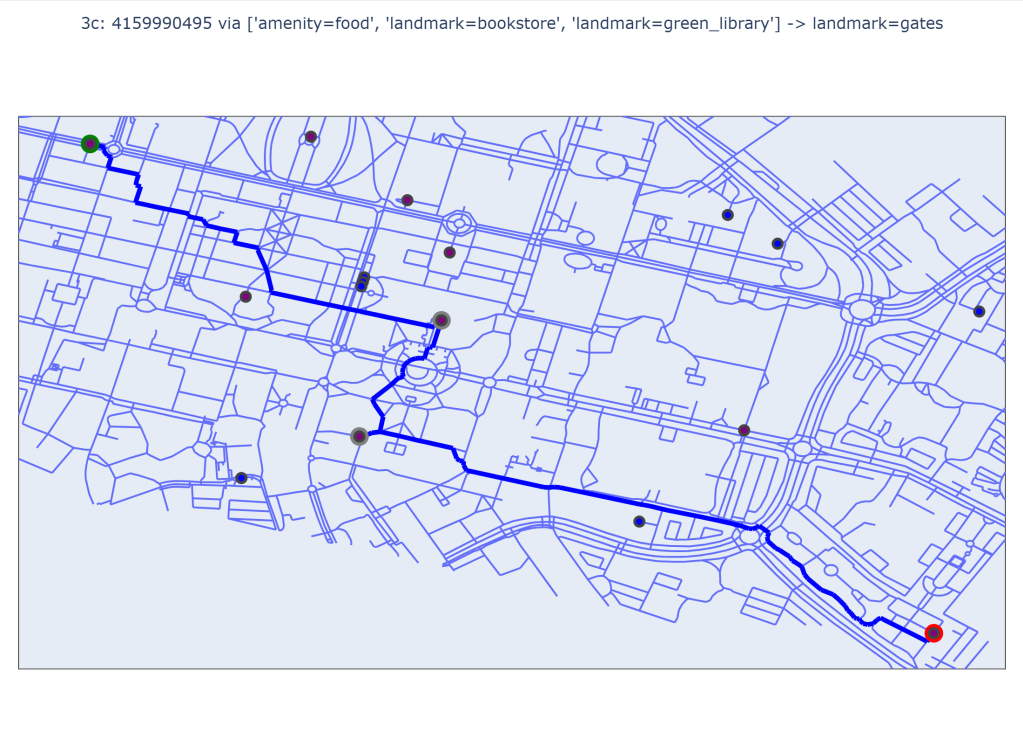


Figure 2: Problem 3c: Rains → {food, bookstore, Green Library} → Gates. Total length 2 256 m over 160 locations; UCS explored 22 557 states (vs. ≈2 000 for the plain shortest-path queries above).

The solver chose the order Rains → Bookstore → Green Library → Gates, 2 256 m in total, and it satisfied the food requirement "for free": Coupa Café sits at the same map node as Green Library, so one location covers two tags and no detour to Tresidder or the Cantor café is needed. That is exactly the kind of shortcut the unordered-waypoint formulation is meant to discover, and it matches what a person who knows the campus would do. Two things the path fails to capture: it treats the Green Library node as "food" even though Coupa's opening hours or a long queue might make Tresidder the better stop, and it minimises distance only, ignoring that the Bookstore closes early, so a feasible plan at 20:00 would have to be a different, longer tour. The state space also grew by a factor of about $2^k = 8$, as predicted in 3b.

(3d) Ride-sharing and dual use

Ride-sharing apps use similar routing and have been criticised for exploitative nudges (e.g. "forward dispatch"). How could unordered waypoints improve the driver experience between a drop-off and the next pick-up, and what driver information would you need? As a dual-use concern, how could a company use the feature to raise profits at the expense of driver health, or what are the downsides of collecting that information?

What we expect: 3–5 sentences: (a) waypoints that improve the driver experience; (b) at least one dimension of *driver identity* (not trip attributes such as time of day) that informs the choice; (c) a harmful use or a downside of the data collection.

Solution. (a) Waypoints that help drivers. Between a drop-off and the next pick-up the dispatcher could route the driver through waypoints such as a rest area or café with parking (`amenity=cafe`, `amenity=toilets`), a fuel or charging station when the tank or battery is low, a place of worship at prayer times, or simply a quiet spot to take a legally required break; because the waypoints are unordered, the driver still gets the shortest tour that covers them before arriving at the next rider. **(b) Driver information needed.** Choosing appropriate waypoints requires information about the driver as a person, not about the trip: e.g. religion (prayer times and places), dietary needs (halal, vegetarian food stops), disability or health conditions (accessible toilets, medication breaks), and preferences about when and how often they rest. **(c) Dual use.** The same feature lets a company maximise profit at the driver's expense: it can insert waypoints that are good for the business (surge-pricing zones, partner petrol stations, areas where the next pick-up is likely) and keep the driver in constant motion, turning the "break" feature into forward dispatch under another name; and collecting religion, health and routine data creates a sensitive profile that can leak, be sold, or be used to discriminate (e.g. assigning fewer rides to drivers who take breaks), so drivers may reasonably refuse to share it.

Problem 4: Speeding up search with A*

*Programming part. The implementation, documentation and test output are in the notebook, section **3.4a–3.4c**.*

- `a_star_reduction`: A* as UCS on modified costs
- `StraightLineHeuristic`
- `NoWaypointsHeuristic`

Assignment 4: Controlling MountainCar

Problem 1: Value iteration

States $= \{-2, -1, 0, +1, +2\}$, start state 0, end states ± 2, Actions $= \{a_1, a_2\}$, $\gamma = 1$. For $i \in \{-1, 0, 1\}$: $T(i{-}1 \mid i, a_1) = 0.8$, $T(i{+}1 \mid i, a_1) = 0.2$; $T(i{-}1 \mid i, a_2) = 0.7$, $T(i{+}1 \mid i, a_2) = 0.3$. Reward$(s, a, s') = 10$ if $s' = -2$; 50 if $s' = +2$; -5 otherwise.

(1a) Two iterations

Give $V_i^\star(s)$ for every s after iterations $i = 1, 2$ (with $V_0^\star \equiv 0$, and $V_i^\star = 0$ at end states).

What we expect: All 10 values.

Solution.

	$s=-2$	$s=-1$	$s=0$	$s=+1$	$s=+2$
$V_1^\star$	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]
$V_2^\star$	[TODO:]	[TODO:]	[TODO:]	[TODO:]	[TODO:]

[TODO: show the Q-value computations]

(1b) Optimal policy

Using $V_2^\star$, what is $\pi^\star(s)$ for each non-end state?

What we expect: $\pi^\star(s)$ for $s \in \{-1, 0, +1\}$.

Solution. [TODO: answer]

Problem 2: Transforming MDPs

(2a) Removing the discount

Given an MDP with discount $\gamma < 1$ and a solver that only handles $\gamma' = 1$, define a new MDP with States$' =$ States $\cup \{o\}$, the same actions, and $\gamma' = 1$. Specify $T'(s' \mid s, a)$ and Reward$'(s, a, s')$ so that $V_{\text{opt}}(s)$ is unchanged for every $s \in$ States.

What we expect: T' and Reward$'$ as formulas, followed by a short verification that the optimal-value recurrences coincide. Use the notation of the question.

Solution. [TODO: answer]

Problem 3: Value iteration on Mountain Car

*Programming part. The implementation, documentation and test output are in the notebook, section **4.3a–4.3b**.*

`value_iteration` and `ModelBasedMonteCarlo`.

(3c) Model-based value iteration: results

Run `python train.py -agent value-iteration` (three trials). Comment on the reward plots, and discuss when model-based value iteration can perform poorly in general.

What we expect: The plots, plus 2–3 sentences describing them and when the method may fail.

Solution. **[TODO: include figures/04_mountaincar/mcvi_*.png and discuss]**

Problem 4: Q-learning Mountain Car

*Programming part. The implementation, documentation and test output are in the notebook, section **4.4a–4.4c**.*

`TabularQLearning`, `fourier_feature_extractor`, `FunctionApproxQLearning`.

(4d) Tabular vs. function approximation

Run `train.py -agent tabular` and `-agent function-approximation`. Compare the plots. Tabular Q-learning is expected to do better on this task. Why might that be?

What we expect: The plots, plus 2–3 sentences comparing the two and giving possible reasons.

Solution. **[TODO: include figures/04_mountaincar/tabular_*.png, fapprox_*.png and discuss]**

(4e) When function approximation wins

Why could function-approximation Q-learning beat tabular Q-learning in other environments, e.g. with limited exploration, very high-dimensional state spaces, or tight memory?

What we expect: 2–3 sentences.

Solution. **[TODO: answer]**

Problem 5: Safe exploration

(5a) `max_speed` in Gym

Explain how `self.max_speed` is used in Gym's `mountain_car.py`.

What we expect: One sentence.

Solution. [TODO: answer]

(5b) Removing the constraint

Run `train.py -agent function-approximation -max_speed=100000`. Why does the behaviour of the learned policy barely change compared with `max_speed` = 0.07?

What we expect: 1–2 sentences.

Solution. [TODO: answer]

(5c) Constrained Q-learning

After implementing `ConstrainedQLearning`, so that only actions with $\text{velocity}_{t+1} < \text{threshold}$ are allowed, with $\text{velocity}_{t+1} = \text{velocity}_t + (\text{action} - 1) \cdot \text{force} - \cos(3\,\text{position}_t) \cdot \text{gravity}$, run `grader.py 5c-helper`. Why do the policies for `max_speed` = 0.065 and an unconstrained `max_speed` differ?

What we expect: 1–2 sentences (only the written answer is graded). Code: notebook §4.5c.

Solution. [TODO: answer]

(5d) Designing safe MDPs for autonomous driving

An autonomous car is trained with RL on public residential roads (pedestrians, vehicles, traffic signals; camera sensing). Design **MDP A** with an exploration policy that could cause harm, and **MDP B**, a modification that reduces or avoids that harm.

What we expect: For each MDP: States, Actions(s), Reward(s, a, s'), an exploration policy, and a 1–2 sentence ethical explanation.

Solution. **MDP A.** [TODO: States / Actions(s) / Reward(s,a,s') / exploration policy / harm]

MDP B. [TODO: States / Actions(s) / Reward(s,a,s') / exploration policy / mitigation]

Assignment 5: Pac-Man

Problem 1: Minimax

(1a) Multi-ghost minimax recurrence

There are $n+1$ agents $a_0, \ldots, a_n$: Pac-Man (a_0, max) and n ghosts (min). One unit of depth means all $n+1$ agents move once. d starts at $d_{\max}$ at the root and decreases. Write $V_{\text{minmax}}(s, d)$ as a piecewise function of IsEnd(s), Utility(s), Eval(s), Player(s), Actions(s) and Succ(s, a); n may be used, $d_{\max}$ may not.

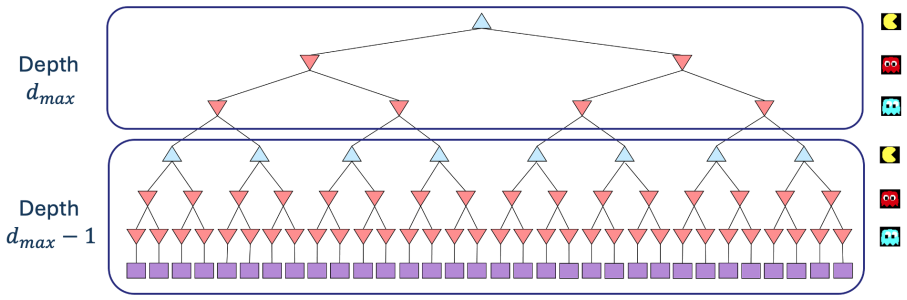


Figure 3: Depth vs. tree height (from the handout).

What we expect: The recurrence as a piecewise function, using exactly these functions.

Solution.

$$V_{\text{minmax}}(s, d) = \begin{cases} \text{Utility}(s) & \text{IsEnd}(s) \\ \text{[TODO:]} & \text{[TODO:]} \end{cases}$$

(1b) `MinimaxAgent`

*Programming part. The implementation, documentation and test output are in the notebook, section **5.1b**.*

Problem 2: Alpha-beta pruning

*Programming part. The implementation, documentation and test output are in the notebook, section **5.2a**.*

Problem 3: Expectimax

(3a) Expectimax recurrence

Write $V_{\text{exptmax}}(s, d)$, the maximum expected utility against ghosts that each pick a legal move uniformly at random, using the same functions as in 1a.

What we expect: A piecewise recurrence in the style of 1a.

Solution. [TODO: answer]

(3b) `ExpectimaxAgent`

*Programming part. The implementation, documentation and test output are in the notebook, section **5.3b**.*

Problem 4: Evaluation function (extra credit)

(4a) `better_evaluation_function`

*Programming part. The implementation, documentation and test output are in the notebook, section **5.4a**.*

(4b) Describing the evaluation function

Describe the evaluation function and its high-level motivation. What else did you try, and what worked and what did not? State whether you consent to the solution being shared if it places in the top 3.

What we expect: A short paragraph.

Solution. [TODO: answer]

Problem 5: AI (mis)alignment and reward hacking

Compare `python pacman.py -p MinimaxAgent -l trappedClassic -a depth=3` with the `ExpectimaxAgent` on the same layout (run each several times).

(5a) Why the agents differ

Why does the minimax agent always rush towards the *closest* ghost, while the expectimax agent (occasionally) does not?

What we expect: One sentence for minimax, stating the assumption that causes this, and one sentence for expectimax.

Solution. [TODO: answer]

(5b) Fixing the objective

Suggest a change to Eval(s) (`score_evaluation_function`) and/or Utility(s) (the final score), or to how the game score is computed, that would stop minimax from dying instantly on `trappedClassic` and make it behave more like expectimax.

What we expect: 1–2 sentences describing the change and why it works.

Solution. [TODO: answer]

(5c) Another real-world example

Describe a realistic scenario outside Pac-Man in which a designer's objective is open to reward hacking or causes negative side effects.

What we expect: 2–5 sentences, stating whether it is reward hacking, a negative side effect, or both, with a justification.

Solution. [TODO: answer]

Assignment 6: Bayesian Networks (Car Tracking)

Set-up: our car's position a_t is observed. The other car's location is C_t (hidden). The microphone reading is $D_t \sim \mathcal{N}(\|a_t - C_t\|_2, \sigma^2)$. Lower-case $p(\cdot)$ denotes local conditional distributions of the Bayesian network; upper-case $\mathbb{P}(\cdot)$ denotes joint or posterior quantities obtained by inference.

Problem 1: Emission probabilities

The other car is stationary: $C_t = C$ for all t.

(1a) Bayesian network

Draw the Bayesian network over $C, D_1, D_2, D_3, a_1, a_2, a_3$, shading the variables observed after $t = 3$.

What we expect: A legible drawing with one arrow from each parent of each variable.

Solution. **[TODO: draw with TikZ: nodes use the styles latent / obs, arrows use edge]**

(1b) Joint distribution

Express $\mathbb{P}(C = c, D_1 = d_1, D_2 = d_2, D_3 = d_3)$ using only local conditional distributions (a_1, a_2, a_3 are known with probability 1).

What we expect: A formula that uses only local conditional distributions.

Solution. **[TODO: answer]**

(1c) `ExactInference.observe`

*Programming part. The implementation, documentation and test output are in the notebook, section **6.1c**.*

Problem 2: Transition probabilities

The other car moves according to $p(c_{t+1} \mid c_t)$, given in code as `self.transProb`.

(2a) Bayesian network

Draw the Bayesian network over $C_1, C_2, C_3, D_1, D_2, D_3, a_1, a_2, a_3$, shading the variables observed after $t = 3$.

What we expect: A legible drawing with one arrow from each parent of each variable.

Solution. [TODO: TikZ drawing]

(2b) Joint distribution

Express $\mathbb{P}(C_1 = c_1, C_2 = c_2, C_3 = c_3, D_1 = d_1, D_2 = d_2, D_3 = d_3)$ using only local conditional distributions.

What we expect: A formula that uses only local conditional distributions.

Solution. [TODO: answer]

(2c) `ExactInference.elapseTime`

*Programming part. The implementation, documentation and test output are in the notebook, section **6.2c**.*

Problem 3: Which car is it?

With K cars, we observe only $\mathbf{E}_t = [E_{t1}, \ldots, E_{tK}]$, a uniformly random permutation of the true noisy distances $\mathbf{D}_t = [D_{t1}, \ldots, D_{tK}]$, so a reading cannot be attributed to a specific car. All distances are distinct.

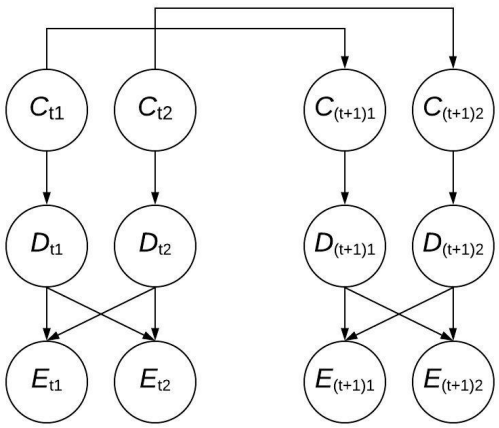


Figure 4: Flow of information for $K = 2$ (from the handout; not a Bayesian network).

(3a) Posterior for $K = 2$, $T = 1$

Write $p(C_{11} = c_{11}, C_{12} = c_{12} \mid \mathbf{E}_1 = \mathbf{e}_1)$ in terms of the Gaussian pdf $p_{\mathcal{N}}(v; \mu, \sigma^2)$ and the priors $p(c_{11}), p(c_{12})$, without D_{11}, D_{12}. (For $K = 1$: $p(c_{11} \mid \mathbf{e}_1) \propto p(c_{11})\, p_{\mathcal{N}}(e_{11}; \|a_1 - c_{11}\|_2, \sigma^2)$.)

What we expect: The expression (proportionality is fine) and the steps taken to derive it.

Solution. [TODO: answer]

(3b) At least $K!$ maximisers

If the prior $p(c_{1i})$ is the same for every car, show that at least $K!$ assignments $(c_{11}, \ldots, c_{1K})$ attain the maximum of $p(C_{11} = c_{11}, \ldots, C_{1K} = c_{1K} \mid \mathbf{E}_1 = \mathbf{e}_1)$ (assume the maximising locations are distinct).

What we expect: A short mathematical argument or a concise explanation.

Solution. [TODO: answer]

Problem 4: Ethics in advanced technologies

(4a) Camera data misuse (dual use)

Self-driving cars often use cameras, which capture faces, licence plates and more. How might camera data or its readings be misused, and what measures could pre-empt the misuse (or why are there none)?

What we expect: 4–6 sentences.

Solution. [TODO: answer]

(4b) Sensor deception

*Programming part. The implementation, documentation and test output are in the notebook, section **6.4b**.*

Observed distance after the attack: $D'_t = \frac{1}{1+\text{skewness}^2} D_t + \sqrt{\frac{2}{1+\text{skewness}^2}}$, implemented in `ExactInferenceWithSensorDeception.observe`.

(4c) Guarding against sensor deception

How could you guard against sensor deception if the type of attack is known in advance? (Hint: sensor redundancy.)

What we expect: 1–2 sentences.

Solution. [TODO: answer]

(4d) Particle filtering vs. exact inference

Run `python drive.py -a -d -k 3 -i particleFilter`. How does the simulation differ from exact inference, and why?

What we expect: 1–2 sentences.

Solution. [TODO: answer]

(4e) Privacy–accuracy trade-off

Particle filtering can run on-device (more private and secure) but is less accurate than cloud-based exact inference. How would you weigh this trade-off?

What we expect: 3–5 sentences.

Solution. [TODO: answer]

(4f) Adversarial attack on camera-based training

A camera-based autonomous vehicle is further trained on customers' anonymised camera data. Name an adversarial attack, its potential impact on the vehicle, and a mitigation.

What we expect: 3–5 sentences.

Solution. [TODO: answer]

Assignment 7: From Language to Logic

Problems 1–5 are programming problems (formulas written with `logic.py`). We list the formulas in mathematical notation so the report can be read on its own; the code and grader output are in the notebook.

Problem 1: Propositional logic

*Programming part. The implementation, documentation and test output are in the notebook, section **7.1**.*

(1a) "If it's summer and we're in California, then it doesn't rain." **[TODO: formula]**

(1b) "It's wet if and only if it is raining or the sprinklers are on." **[TODO: formula]**

(1c) "Either it's day or night (but not both)." **[TODO: formula]**

Problem 2: First-order logic

*Programming part. The implementation, documentation and test output are in the notebook, section **7.2**.*

(2a) "Every person has a parent." **[TODO: formula]**

(2b) "At least one person has no children." **[TODO: formula]**

(2c) Define Father(x, y) in terms of Male and Parent. **[TODO: formula]**

(2d) Define Granddaughter(x, y) in terms of Female and Child. **[TODO: formula]**

Problem 3: Liar puzzle

*Programming part. The implementation, documentation and test output are in the notebook, section **7.3a**.*

Suspects: John, Susan, Mark, Nicole. Mark: "It wasn't me!" John: "It was Nicole!" Nicole: "It wasn't me, it was Susan!" Susan: "Nicole's a liar." Exactly one person tells the truth, and exactly one person crashed the server.

[TODO: the six formulas and the answer returned by `grader.py 3a-run`]

Problem 4: Odd and even integers

*Programming part. The implementation, documentation and test output are in the notebook, section **7.4a**.*

[TODO: the six axioms in mathematical notation and the result of `grader.py 4a-run`]

Problem 5: Semantic parsing (extra credit)

*Programming part. The implementation, documentation and test output are in the notebook, section **7.5**.*

Grammar rules for: *every $Noun $Verb some $Noun*; *there is some $Noun that every $Noun $Verb*; *if a $Noun $Verb a $Noun then the former $Verb the latter.*

Problem 6: Explainability of logic-based systems

The four considerations: *respect*; *assessing the fairness of rules*; *contesting and correcting decisions*; *ability to change user behaviour*. Context: the liar puzzle of Problem 3, where a conclusion could have real consequences (e.g. someone being fired).

(6a) What counts as an explanation

Besides the final answer, what does a general first-order logic system, like the one used in Problem 3, return that could count as an explanation?

What we expect: 1–2 sentences.

Solution. [TODO: answer]

(6b) Why it may not be adequate

Why might this explanation not be adequate? (Hint: model misspecification, or how the question is framed.)

What we expect: 1–2 sentences.

Solution. [TODO: answer]

(6c) Which considerations are met

Are any of the four explainability considerations met by the explanations a logic-based system produces?

What we expect: 2–4 sentences.

Solution. [TODO: answer]

Problem 7: Applications of soundness and completeness

Sound: $\{f : KB \vdash f\} \subseteq \{f : KB \models f\}$. Complete: $\{f : KB \vdash f\} \supseteq \{f : KB \models f\}$.

(7a) Is an LLM sound or complete?

Treat whatever GPT-3.5 generates as what it "derives". Given that LLMs hallucinate, is GPT-3.5 sound, complete, both or neither? Use examples and the definitions.

What we expect: 2–3 sentences for each property (4–6 in total).

Solution. [TODO: answer]

(7b) Safety-critical deployment

System A is sound but not complete, and System B is complete but not sound. Which would you deploy in a safety-critical setting, where acting on a false conclusion has severe consequences, and why?

What we expect: 2–3 sentences tied to the definitions.

Solution. [TODO: answer]

(7c) With a separate verifier

With a separate verifier available (e.g. a human in the loop) and the goal of capturing as much true information as possible, which system would you choose, and why?

What we expect: 2–3 sentences tied to the definitions.

Solution. [TODO: answer]